

[5회차] ML 과제 — 신용카드 사기 탐지 Report

YBIGTA Newbie Assignment · 5(1)-Basic_ML · 작성자: 권준범 · 2026-07-30

Kaggle Credit Card Fraud Detection 데이터셋을 이용해 클래스 불균형 상황에서 사기 거래를 탐지하는 분류 모델을 구축했다. 데이터 탐색 → 샘플링 → 전처리 → SMOTE 오버샘플링 → 모델 학습 → 하이퍼파라미터/threshold 튜닝 순으로 진행했으며, 상세 코드와 실행 결과는 `basic_ML.ipynb`에 담겨 있다.

1. 데이터 로드 및 기본 탐색

원본 데이터는 총 284,807건이며, 30개 특성(Time, V1~V28, Amount)과 타깃(Class)으로 구성된다. V1~V28은 PCA로 이미 익명화·스케일링된 변수이고, Time과 Amount만 원본 스케일이다.

구분	건수	비율
정상 거래 (Class=0)	284,315	99.83%
사기 거래 (Class=1)	492	0.17%

사기 거래가 전체의 0.17%에 불과한 극단적인 클래스 불균형 데이터셋임을 확인했다.

2. 샘플링

사기 거래(Class=1) 492건은 전부 유지하고, 정상 거래(Class=0)는 `random_state=42`로 10,000건만 무작위 샘플링해 결합했다.

구분	건수	비율
정상 거래	10,000	95.31%
사기 거래	492	4.69%
합계	10,492	100%

샘플링으로 데이터 규모를 줄이면서도 사기 비율을 0.17% → 4.69%로 크게 완화했다.

3. 데이터 전처리

`Amount` 변수만 `StandardScaler`로 표준화해 `Amount_Scaled`로 대체하고 원본 `Amount`는 제거했다. V1~V28은 이미 스케일링되어 있어 추가 처리가 불필요했다. 이후 X(30개 피처), y(Class)로 분리했다.

4. 학습/테스트 데이터 분할

`train_test_split(test_size=0.2, stratify=y, random_state=42)`로 8:2 분할했다. `stratify` 옵션으로 분할 후에도 클래스 비율이 원본과 거의 동일하게 유지됨을 확인했다.

구분	정상(0)	사기(1)	사기 비율
Train	7,999	394	4.69%
Test	2,001	98	4.67%

5. SMOTE 적용

왜 SMOTE를 적용해야 하는가

샘플링 이후에도 학습 데이터의 사기 비율은 약 4.7%로 여전히 불균형하다. 이 상태로 학습하면 모델이 다수 클래스(정상)만 잘 맞혀도 높은 accuracy를 얻을 수 있어, 소수 클래스(사기)의 패턴을 충분히 학습하지 못하고 실제 사기를 정상으로 오분류(False Negative)하는 경향이 커진다. 사기 탐지 문제는 사기를 놓치는 비용이 훨씬 크므로 Recall이 핵심 지표인데, 단순 accuracy 최적화는 이 목적과 충돌한다.

SMOTE(Synthetic Minority Over-sampling Technique)는 소수 클래스 샘플 사이를 보간해 현실적인 합성 샘플을 생성함으로써 소수 클래스 데이터를 늘린다. 단순 복제(Random Oversampling)와 달리 새로운 샘플을 만들어내므로 모델이 소수 클래스의 결정 경계를 더 잘 학습하도록 돕고, 과적합 위험도 상대적으로 낮다. 단, SMOTE는 반드시 **학습 데이터에만** 적용하고 테스트 데이터는 원본 분포를 유지해야 실제 성능을 왜곡 없이 평가할 수 있다.

구분	정상(0)	사기(1)
SMOTE 적용 전	7,999	394
SMOTE 적용 후	7,999	7,999

6. 모델 학습

LogisticRegression과 RandomForest 두 모델을 비교했다 (둘 다 SMOTE로 오버샘플링된 학습 데이터로 학습, 테스트셋은 원본 분포 그대로 평가).

모델	Class	Precision	Recall	F1	PR-AUC
LogisticRegression	0 (정상)	0.9970	0.9895	0.9932	0.9523
	1 (사기)	0.8142	0.9388	0.8720	
RandomForest (threshold=0.5)	0 (정상)	0.9945	0.9975	0.9960	0.9537
	1 (사기)	0.9457	0.8878	0.9158	

LogisticRegression은 Class 1(사기) F1-score가 0.8720으로 목표(0.88)에 못 미친 반면, RandomForest는 threshold=0.5 기준으로 이미 세 목표($\text{Recall} \geq 0.80$, $\text{F1} \geq 0.88$, $\text{PR-AUC} \geq 0.90$)를 모두 상회했다. 비선형 결정 경계를 학습할 수 있고 SMOTE로 생성된 합성 샘플에도 안정적으로 동작해 **RandomForest를 최종 모델로 선정했다**.

7. 최종 성능 평가

하이퍼파라미터 비교

모델	PR-AUC
RandomForest (n_estimators=100, default)	0.9537
RandomForest (n_estimators=200, max_depth=10)	0.9517

단순 depth 제한이나 estimator 수 증가는 오히려 성능을 소폭 낮췄다. 기본 설정(n_estimators=100)이 가장 우수해 이를 최종 모델로 유지했다.

Threshold 조정

0.10~0.90 범위에서 0.05 간격으로 threshold를 조정하며 두 클래스 모두에서 목표($\text{Recall} \geq 0.80$, $\text{F1} \geq 0.88$)를 만족하는 구간을 탐색했다. threshold가 낮을수록 사기(Class 1) Recall은 높아지지만 Precision과 F1이 급격히 낮아지고, threshold가 너무 높으면 사기 Recall이 목표 아래로 떨어졌다. 목표를 만족하는 구간은 threshold 0.45~0.85였으며, 두 클래스의 F1 합이 최대가 되는 **threshold=0.65**를 최종 값으로 선정했다.

최종 결과

Class	Precision	Recall	F1
0 (정상)	0.9930	0.9995	0.9963
1 (사기)	0.9882	0.8571	0.9180

PR-AUC (average_precision_score) = **0.9537**

목표 달성 여부

지표	목표	Class 0	Class 1	달성 여부
Recall	≥ 0.80	0.9995	0.8571	달성
F1-score	≥ 0.88	0.9963	0.9180	달성
PR-AUC	≥ 0.90	0.9537		달성

→ Class 0, Class 1 모두 세 가지 목표(Recall, F1, PR-AUC)를 전부 달성했다.

추가 개선 방향 (참고)

목표는 이미 달성했지만, 성능을 더 끌어올리고 싶다면 다음을 시도해볼 수 있다.

- XGBoost/LightGBM 등 부스팅 계열 모델과 `scale_pos_weight` 옵션 활용
- GridSearchCV/Optuna를 통한 폭넓은 하이퍼파라미터 탐색 (max_depth, min_samples_leaf, n_estimators 등)

3. SMOTE 외 ADASYN, BorderlineSMOTE 등 다른 오버샘플링 기법과의 비교
4. Time 변수를 시간대별 파생 변수로 가공하는 등의 feature engineering
5. Precision-Recall Curve를 바탕으로 오탐(False Positive)과 미탐(False Negative)의 실제 비즈니스 비용을 반영한 threshold 최적화

전체 코드와 실행 결과(각 셀의 출력 포함)는 basic_ML.ipynb에서 확인할 수 있다.